

Desktop.Space

Intelligente Desktop-Kontexterkennung und -Automation mit Eye-Tracking, Messaging-Pipeline und Moire-Vision.

Overview

Desktop.Space gibt Vibemind Augen und Hände auf dem Desktop des Users. Der Space operiert über mehrere integrierte Systeme: Eye/Gaze-Tracking via Webcam (MediaPipe), Screen-Analyse, automatisierte Maus/Tastatur-Steuerung, und eine Messaging-Pipeline für Chat-Automation. Das eyeterm-Subsystem allein umfasst 15 Subdirectories.

Backend-Agent: DesktopAgent (21 Events)

Datei: `python/spaces/desktop/agents/desktop_agent.py`

Desktop-Operationen (7 Events)

Event	Tool-Funktion	Beschreibung
<code>desktop.open_app</code>	<code>open_app</code>	Anwendung öffnen
<code>desktop.click</code>	<code>click_element</code>	Element klicken
<code>desktop.type</code>	<code>type_text</code>	Text eingeben
<code>desktop.press_key</code>	<code>press_key</code>	Taste drücken
<code>desktop.screenshot</code>	<code>take_screenshot</code>	Screenshot erstellen
<code>desktop.scroll</code>	<code>scroll_screen</code>	Scrollen
<code>desktop.task</code>	<code>execute_desktop_task</code>	Desktop-Task ausführen

Task-Management (3 Events)

Event	Tool-Funktion
<code>desktop.task.create</code>	<code>create_task_node</code>
<code>desktop.task.update</code>	<code>update_task_status</code>
<code>desktop.task.list</code>	<code>get_task_list</code>

Moire Vision (2 Events)

Event	Tool-Funktion
<code>desktop.moire.scan</code>	<code>moire_scan</code>
<code>desktop.moire.find</code>	<code>moire_find_element</code>

Messaging / Clawdbot-Bridge (7 Events)

Event	Tool-Funktion
<code>messaging.whatsapp</code>	<code>send_whatsapp</code>
<code>messaging.telegram</code>	<code>send_telegram</code>
<code>messaging.send</code>	<code>send_message</code>
<code>web.search</code>	<code>web_search</code>
<code>web.fetch</code>	<code>web_fetch</code>
<code>openclaw.status</code>	<code>get_clawdbot_status</code>
<code>openclaw.notifications</code>	<code>get_notifications</code>

eyeterm-Steuerung (2 Events)

Event	Tool-Funktion
<code>desktop.eyeterm.toggle</code>	<code>eyeterm_toggle</code>
<code>desktop.eyeterm.calibrate</code>	<code>eyeterm_calibrate</code>

eyeterm-Subsystem (15 Directories)

Vollständiges Eye-Tracking-System unter `python/spaces/desktop/eyeterm/`:

Directory	Zweck
<code>vision/</code>	Gaze-Tracking (gaze.py 21.2KB), Wink-Detection (wink.py 4.8KB), PolynomialMapper (9.2KB), Camera, Calibrate
<code>action/</code>	Action-Execution
<code>ai/</code>	AI-basierte Entscheidungen
<code>audio/</code>	Audio-Integration
<code>claude/</code>	Claude-API Integration
<code>cursor/</code>	Gaze-to-Cursor Mapping
<code>editor/</code>	Editor-Integration
<code>models/</code>	Datenmodelle
<code>routing/</code>	Command-Routing
<code>screen/</code>	Screen-State-Analyse
<code>stream/</code>	Stream-Verarbeitung
<code>ui/</code>	UI-Rendering
<code>app.py</code>	Haupt-App (22.8KB)
<code>headless.py</code>	Headless-Modus (38.6KB)

Directory	Zweck
state.py	State-Management (5.4KB)

Vision-Stack im Detail

- **gaze.py** (21.2KB): MediaPipe FaceMesh (478-Landmark Model), Iris-basiertes Gaze-Tracking, Head-Pose aus Nase/Ohren
- **wink.py** (4.8KB): Eye Aspect Ratio (EAR) Berechnung, Links=Confirm / Rechts=Cancel, 600ms Cooldown
- **polynomial_mapper.py** (9.2KB): Quadratische Gaze-to-Screen Koordinaten-Mapping, Click-Learning Kalibrierung, Adaptive Fusion (Iris+Head)

Messaging-Pipeline

Vollständige Messaging-Verarbeitung unter `python/spaces/desktop/messaging/`:

Datei	Zweck
incoming_handler.py (8.8KB)	Eingehende Nachrichten verarbeiten
messaging_pipeline.py (11.9KB)	Pipeline-Orchestrierung
relevance_filter.py (7.3KB)	Relevanz-Filterung

Key Components

Komponente	Datei	Zweck
Backend-Agent	agents/desktop_agent.py	21 Events → 19 Tools
Desktop-Tools	tools/desktop_tools.py	Maus/Tastatur-Automation
Task-Tools	tools/task_tools.py	Task-Node-Management
Moire-Tools	tools/moire_tools.py	MoireTracker Vision
Adapted Tools	tools/adapted_desktop_tools.py	Typisierte Wrapper
Automation UI	Automation_ui/ (Submodul)	FastAPI Backend für Vision-Agents

Current Status

Implementiert

- Eye/Gaze-Tracking via MediaPipe voll funktional (478-Landmark, Iris+Head)
- PolynomialMapper für Gaze-to-Screen Mapping (quadratisch, click-learning)
- Wink-Detection (EAR-basiert, Links/Rechts-Unterscheidung)
- Kamera-Integration und Kalibrierung
- 21 Events mit vollständigem Tool-Mapping

- Messaging-Pipeline mit Relevance-Filtering (WhatsApp, Telegram)
- Moire Tracker Vision-Integration
- Desktop-Automation (Maus, Tastatur, App-Steuerung)
- Task-Node-Management
- Automation UI (FastAPI Backend)
- eyeterm App mit 15 Subsystemen
- OCR-Engine für Text-Extraktion aus Screen-Captures
- Video-Agent für kontinuierliche Action-Verifizierung
- ML-basiertes Workflow-Learning (Eye-Tracking-Daten als Grundlage)
- Agent-Team für Screen-Validierung

Roadmap

- Multi-monitor support and complex desktop layouts
- Privacy-preserving facial analysis (on-device processing)
- Erweiterte Workflow-Automation basierend auf gelernten User-Patterns
- Cross-Application Context-Sharing zwischen Desktop-Anwendungen

Ecosystem-Fit

Desktop.Space ist Vibeminds Augen und Hände auf dem Computer des Users. Es liefert Kontext an Ideas.Space über den aktuellen Screen-Inhalt. Es erkennt User-Aktivitätsmuster für The Brain.Space. Es führt Aktionen aus, die andere Spaces orchestrieren. Es validiert, dass Deployments von Coding.Space korrekt funktionieren. Die gesamte Vibemind-Plattform wird kontextbewusst durch Desktop.Space.